CSC 251 Lecture 03: Conditionals, Booleans, Iteration

Conditional Statements

Single alternative decision structure:

```
if <condition>:
    statement1
    statement2
statement3
```

Dual alternative decision structure:

```
if <condition>:
    statements or pass
else:
    other statements
yet more statements
```

Relational operators:

`>` `>=` `<` `<=` `==` `!=`

Shorthand if-else (Ternary Operator/Conditional)

```
<expression1> if <condition> else <expression2>
```

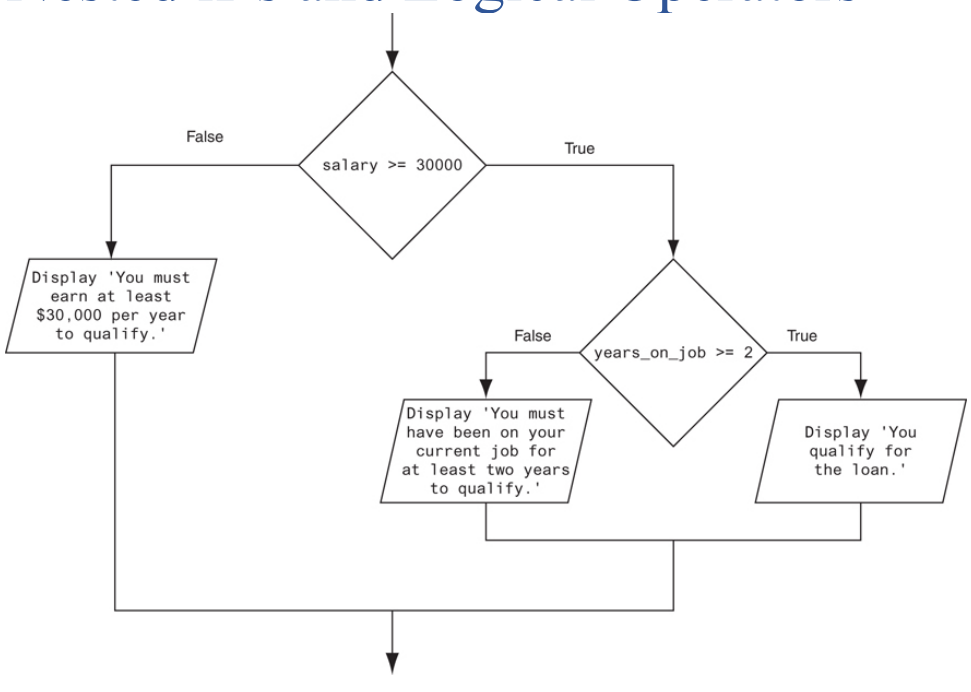
Example 1a:

```
hours = 17
print( "full-time" if hours >= 12 else "part-time" )
```

Example 1b:

```
m = a if a > b else b
```

Nested if's and Logical Operators



Example 2a:

```
MIN_SALARY = 30000
MIN_YEARS = 2
salary=float(input("Enter annual salary: "))
years= int(input("Enter years employed: "))

if salary >= MIN_SALARY:
    if years >= MIN_YEARS:
        print("You quality for the loan.")
    else:
        print("Insufficient years of "
              "employment.")
else:
    print("Insufficient annual salary."
```

How do we know which `if` is paired with each `else`?

indentation

Boolean Logic

Compound Logic: `and`

Example 1c:

optional

```
if (salary >= MIN_SALARY and years >= MIN_YEARS):
    print("You qualify…")
. . .
```

Example 2a:

```
if credit_hours >= 29 and credit_hours < 60:
    print("sophomore standing")
```

Example 2b:

Does a "between" statement work?

```
if 29 <= credit_hours < 60:
    print("sophomore standing")
```

included

Example 2c: More preferred:

```
if credit_hours in range(29,60):
    . . .
```

not included

Compound Logic: `or`

Example 3a:

```
if credit_hours < 29 or credit_hours >= 60:
    print("not sophomore standing")
```

Short-circuit evaluation

Logic: `not`

```
if not(GPA < 2.00):
    print("eligible for graduation")
```

Example 3b: More preferred:

```
if GPA >= 2.00:
    . . .
```

Example 4: Is the following condition true or false?

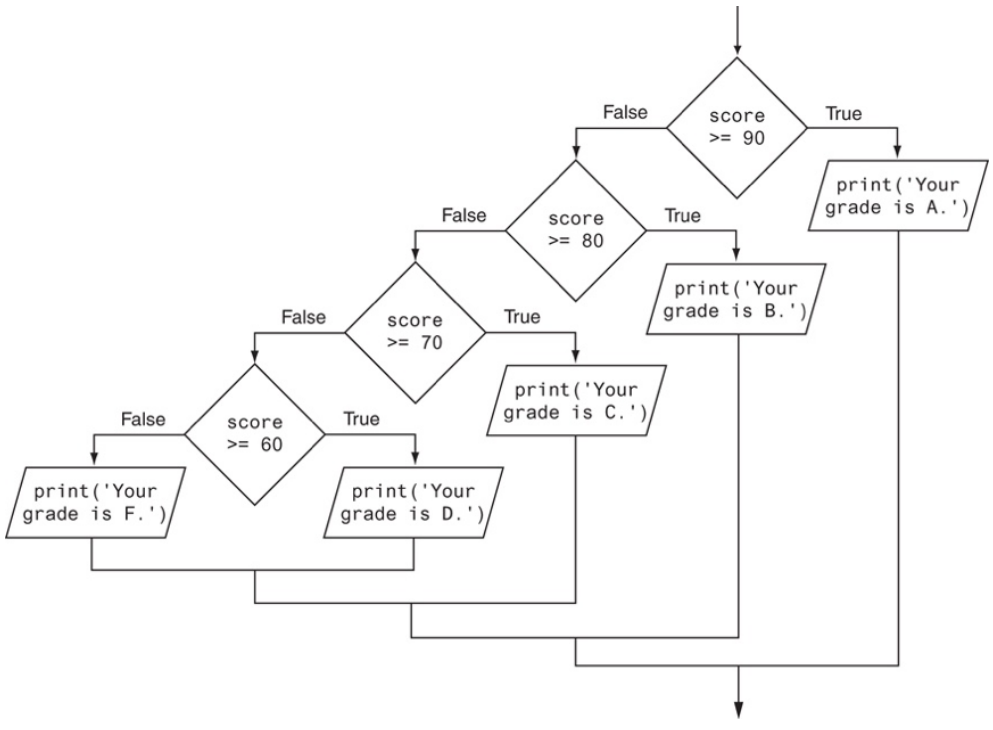
```
x = 8
y = 7
x <= y or not(y > x + 2 and x == y + 2)
```

F or not(F) = F or T = T

Order of operations?

unary +/-
( )
**
* / // %
+ -
relational
logical (not and or)

For a series of decisions: `elif`




```
if <condition1>:
    statements
elif <condition2>:
    other statements
elif <condition3>:
    more statements
. . .
else:
    yet more statement
```

Example 5a: Using nested `if`'s:

```
score = int(input("Enter your test score: "))
if score >= 90:
    print("Your grade is A.")
else:
    if score >= 80:
        print("Your grade is B.")
    else:
        if score >= 70:
            print("Your grade is C.")
        else:
            if score >= 60:
                . . .
```

indentation nightmare

Example 5b: Using `if-elif-else`:

Advantages of `elif`?

```
score = int(input("Enter your test score: "))
if score >= 90:
    print("Your grade is A.")
elif score >= 80:
    print("Your grade is B.")
elif score >= 70:
    print("Your grade is C.")
elif score >= 60:
    print("Your grade is D.")
else:
    print("Your grade is F.")
```

cleaner

Boolean Variables

When should you use Boolean variables? Boolean variables can be assigned a value of True or False:

```
match = False
even = True
```

⇒ 0 (off)
anything but 0 (on)

Example 6:

```
NAME = "Smith"
found = False

last = input("Enter last name: ")
if last == NAME:
    found = True
```

← found == True

```
if found:
    print("Last name: {name}")
else:
    print("Last name is not {name}")
```

Iteration: The while Loop – a condition-controlled loop

A *pretest* loop that tests the condition prior to performing an iteration.

```
while condition:
    statement1
    statement2
```

Example 7:

Output? How many times does the loop iterate?

```
i = 0
N = 15
while i < N:
    print(i, end = " ")
    i = i + 5
```

0 5 10 3X

Example 8:

fudge factor

```
playAgain = 'y'
while playAgain == 'y':
    <game code here>
    playAgain = input("Want to play again?  Enter y for yes: ")

print("Thanks for playing!")
```

Practice

Example 9a:

```
j = 0
while j < 10:
   print(j)
```

∞

Example 9b:

```
k = 5
while k < 5:
   print(k)
   k = k + 2
```

no output - never executed!

Example 9c:

```
n = 5
while n < 8:
   print("executing loop")
   n = n + 1
   if n % 2 == 0:
      break
else:
   print("executing else")
```

executing loop (once)

← only when loop terminates naturally

Example 9d: Does Python have a do-while loop construct?

No, but can emulate

```
x = 0
while True:
   x += 1
   if x < 10:
      continue
   else:
      break
```

continue to top of loop

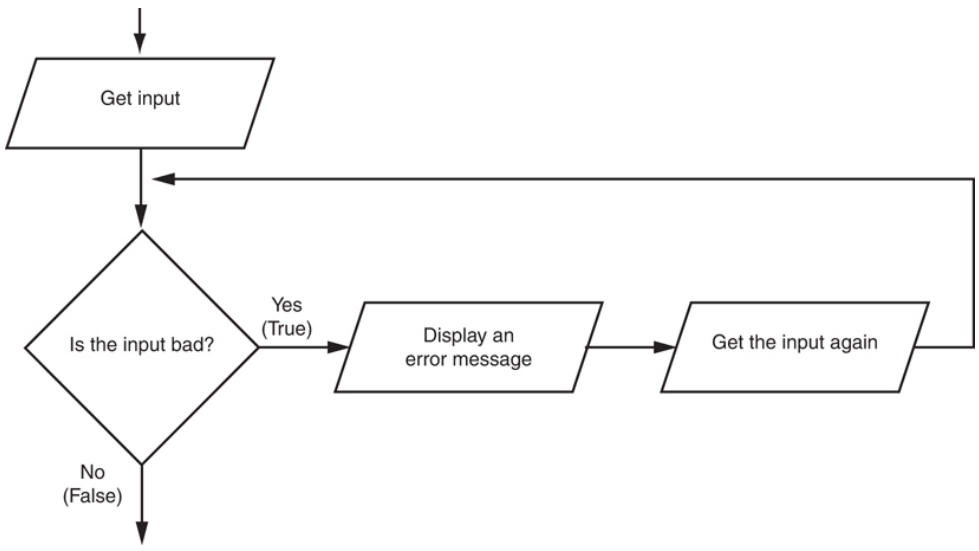
break out of loop

Since we have break and continue, do we have `goto` in Python?



Input Validation

A best practice is to inspect data that is input into a program to make sure it is valid. A while loop can assist in error handling, allowing the user to re-enter the value until it is valid.



Example 10: Input validation for the loan qualifier program

```
salary=float(input("Enter annual salary: "))
years= int(input("Enter years employed: ")
```

while salary < 0:
    salary = float(input ...)

while years < 0 or years > 100:
    years = int(input ...)

For now: Assume all input is of correct type but needs to be validated for range.
Exception handling coming soon!

Iteration: The `for` Loop – a count-controlled loop

Example 11a: Iterating over a sequence of values:

```
for num in [1,2,3,4]:
    print(num, end = " ")
```

Output:

1 2 3 4

where num is the **target variable.**

Example 11b: Let's pick some different variables:

```
for value in [ 100, 3.14, 'go' ]:
    print(2*value, end = " ")
```

200
6.28
gogo

Example 11c: Iterating over a list of names:

```
for name in [ 'Oscar', 'Victor', 'Damon' ]:
    print(name)
```

Oscar
Victor
Damon

Example 12a: Using a **range function** as an iterable object:

```
for val in range(3):
   print(val, "first")
```

Example 12b: or

```
for val in range(0, 3):
   print(val, "a second way…")
```

Example 12c: or

```
for val in range(0, 3, 1):
   print(val, "include a step")
```

Example 12d: effectively the same as:

```
for val in [0,1,2]:
   print(val)
```

same # of iterations

Example 13: Steps can be positive or negative integers. Output?

```
for val in range(3, 0, -1):
   print(val, end = " ")
```

Example 14: Letting the user control the number of iterations:

```
times = int(input("Enter how many times to print out 'Gotcha!'"))
for num in range(1, ____times+1____):
    print("Gotcha!")
```

Loop Practice (see Lect03Code.ipynb)

Example 15a:

```
for m in range(5):
   print(m)
   m += 3
```

0 1 2 3 4

changes only inside loop, assigned next value during next iteration

Example 15b:

```
for i in range(1,5,-2):
   print(i)
```

no output, range object cannot be created with these parameters

Example 16: Calculating a running total using augmented assignment operators

```
MAX = 12
sum = 0
for score in range(MAX):
    quiz = float(input(f"Enter quiz grade {score + 1}: "))
    sum += quiz
print(f"Average = {sum/MAX: .2f}")
```

0 through MAX-1

offsets by 1, 1 through MAX

Sentinels

What are they? When do we use them? How do you choose an appropriate sentinel?

Example 17:

a flag to indicate end of input, often -1, 0, 999...

```
sum = 0
count = 0
quiz = float(input("Enter quiz grade or -1 to end: "))
while quiz != -1:
    sum += quiz
    count += 1
    quiz = float(input("Enter next quiz grade or -1 to end: "))
print(f"Average = {sum/count:.2f}")
```

skips loop when -1 encountered so -1 is not included in calc

What if sum and/or count are not initialized? (try it!)

Nested Loops

Example 18: Consider a digital clock that displays hours, minutes, and seconds:

Output:

```
00:00:00
00:00:01
00:00:02
…
00:00:58
00:00:59
00:01:00
00:01:01
00:01:02
…
00:59:58
00:59:59
01:00:00
01:00:01
…
23:59:59
```

Code:

```
for hours in range( 24 ):

    for minutes in range( 0,60 ):


        for seconds in range(0,60,1 ):

            print(f"{hours:02}:{minutes:02}:{seconds:02}")
```

↑ 2 place values, fills in 0's